

```
=====
TXT2PDF v1.6 - THE ESPARTAN PDF GENERATOR
Zero-dependency TXT to PDF converter in pure C11
=====
```

```
"Turn plain text into structured PDFs without asking for permission."
Language:  C11 (pure, no C++ runtime, no external libs)
Binary:    Static, stripped, musl libc, ~58 KB
Output:    PDF 1.4 with Outlines (Bookmarks) and Internal Links
Status:    Production-ready
```

```
=====
QUICK START
=====
```

```
- Compile dynamically:
gcc -O2 -Wall -Wextra -std=c11 -o txt2pdf txt2pdf.c

- Compile statically with musl:
musl-gcc -O2 -Wall -Wextra -std=c11 -static -o txt2pdf txt2pdf.c

- Run:
./txt2pdf readme.txt readme.pdf

- Run with debug diagnostics:
./txt2pdf --debug readme.txt readme.pdf
```

```
=====
[TOC] TABLE OF CONTENTS
=====
```

1. what is txt2pdf?
2. updated parser rules
3. table of contents rules
4. usage and debug mode
5. minimal working structure
6. pdf internals
7. project structure
8. building and static cookbook
9. known limitations
10. troubleshooting
11. design philosophy
12. license and credits

```
=====
1. WHAT IS TXT2PDF?
=====
```

txt2pdf is a zero-dependency command-line utility that converts plain text files into structured PDF 1.4 documents.

Unlike Pandoc, wkhtmltopdf, or Python-based converters that require hundreds of megabytes of runtime dependencies, txt2pdf is a small static binary.

It parses text, calculates page breaks, generates PDF objects, and writes the final binary stream directly to disk using only the C standard library.

It is designed for environments where installing dependencies is impossible, unwanted, or where the network is down.

```
=====
2. UPDATED PARSER RULES
=====
```

The parser creates PDF bookmarks only for lines that are real section headers.

A line is considered a section header only if all of these conditions are true:

- It starts with a digit, optionally followed by letters, then a dot.
- The title text after the dot contains only uppercase letters.
- The line is surrounded by separator lines made mostly of '=' characters.
- The line is not inside a contents block.

Valid section shape:

- separator line
- uppercase numbered title
- separator line

Rejected lines:

- numbered titles containing lowercase letters
- numbered titles used as examples inside normal paragraphs
- numbered titles not surrounded by separator lines
- numbered titles inside the contents block

This makes the parser much safer for manuals that contain examples.

3. TABLE OF CONTENTS RULES

Internal links are created only from contents blocks.

A contents block is detected when:

- a line contains the token "[TOC]" (e.g., "[TOC]" or "[TOC] TABLE OF CONTENTS")
- the next non-blank line is a separator line
- the block ends at the next separator line

Rules for entries:

- entries must be numbered lines
- lowercase entries are recommended
- entries are not converted into bookmarks
- entries are linked to the matching section by number or title
- uppercase entries inside the contents block are also ignored as bookmarks

This means the old duplicate-bookmark problem is largely eliminated by the parser itself.

4. USAGE AND DEBUG MODE

Basic usage:

```
./txt2pdf input.txt output.pdf
```

Debug usage:

```
./txt2pdf --debug input.txt output.pdf
```

Debug mode prints diagnostics to stderr.

It reports:

- detected encoding (UTF-8 vs Latin-1/ISO-8859-1)
- lines marked as part of a contents block

- detected section headers
- skipped uppercase numbered lines
- detected internal links
- destination page for each section and link

Use debug mode when:

- a bookmark is missing
- too many bookmarks appear
- a contents entry does not link anywhere
- you are adapting a new readme to the espartan format

=====

5. MINIMAL WORKING STRUCTURE

=====

A minimal compatible document needs three things:

- a contents block starting with the [TOC] token
- lowercase or mixed-case contents entries
- uppercase section headers surrounded by separator lines

Recommended structure:

- write the contents title line containing [TOC] (e.g., "[TOC] TABLE OF CONTENTS")
- write a separator line
- write entries such as "1. introduction" and "2. conclusion"
- write a separator line
- for each section, write a separator line
- write the uppercase header, for example "1. INTRODUCTION"
- write another separator line
- write the section body

Important:

Do not wrap example headers with separator lines unless you want them to become real bookmarks.

=====

6. PDF INTERNALS

=====

txt2pdf generates a valid PDF 1.4 structure from scratch.

Catalog and Pages:

Calculates A4 dimensions, applies margins, wraps text using Courier 9pt font with 12pt leading.

Encoding & Smart Decoder:

Auto-detects UTF-8 vs Latin-1. If UTF-8 is detected, a smart decoder translates multi-byte sequences to WinAnsiEncoding on the fly, ensuring perfect rendering of accents (áéíóú), ñ, ü, and their uppercase variants.

Outlines:

Generates a linked list of PDF Outline objects pointing to the exact XYZ coordinates of each uppercase header.

Annotations:

Scans contents blocks and injects link annotations for each entry matching a real section.

Cross-Reference Table:

Calculates byte offsets for every object and writes a compliant xref table and trailer.

=====

7. PROJECT STRUCTURE

=====

The txt2pdf codebase follows a single-file architecture.

Typical layout:

- txt2pdf.c
- Makefile, optional
- readme.txt, optional

The main C file contains:

- argument parsing
- encoding detection
- text parsing
- header detection
- contents detection
- page break calculation
- PDF object generation
- binary stream writing

The tool is intentionally small and auditable.

=====

8. BUILDING AND STATIC COOKBOOK

=====

Dynamic build:

```
gcc -O2 -Wall -Wextra -std=c11 -o txt2pdf txt2pdf.c
```

Static musl build:

```
musl-gcc -O2 -Wall -Wextra -std=c11 -static -o txt2pdf txt2pdf.c
```

Why musl?

glibc static binaries are often much larger due to NSS and locale data. musl provides a clean, minimal libc that results in a tiny, fully self-contained executable.

Verify static build:

```
file txt2pdf
```

```
ldd txt2pdf
```

Expected:

- file reports statically linked
- ldd says it is not a dynamic executable

=====

9. KNOWN LIMITATIONS

=====

- Font is hardcoded to Courier.
- WinAnsiEncoding only (UTF-8 is dynamically translated to WinAnsi).
- Characters outside WinAnsi are replaced with '?'.
- No images or vector graphics.
- No external URL hyperlinks.
- No custom page sizes.
- No custom margins.
- No multi-column layout.
- Section headers must be fenced by separator lines.
- Internal links are only generated from contents blocks.

=====

10. TROUBLESHOOTING

=====

Missing bookmarks:

- The header contains lowercase letters.
- The header is not surrounded by separator lines.

- The header is accidentally inside a contents block.

Duplicate bookmarks:

- Old documents may have uppercase contents entries.
- The updated parser ignores contents entries as sections.
- If duplicates persist, run with --debug and inspect skipped lines.

Missing links:

- The entry is not inside a detected contents block.
- The entry number does not match any section.
- The entry title does not match any section title.

Extra bookmarks:

- An example header is surrounded by separator lines.
- Remove the separators around examples or rewrite the example line.

Garbled characters (accents, ñ):

- Ensure your text editor saves the file in UTF-8.
- Run with --debug to verify the detected encoding.

Debug output is the fastest way to diagnose the problem.

=====

11. DESIGN PHILOSOPHY

=====

Modern software development has forgotten how to build small tools.
We wrap simple text transformations in Electron apps, Node.js runtimes,
and Docker containers.

txt2pdf is a rejection of that bloat.

It proves that a fully functional, structured document generator can fit
in a tiny static binary, written in pure C11, with zero external
dependencies.

When the cloud goes down, and the package managers fail to resolve,
the espartan tool remains.

"Code is neither created nor destroyed; it is distilled."

=====

12. LICENSE AND CREDITS

=====

License:

GPLv3.

Credits:

txt2pdf was written as a demonstration of minimalist software design.

Inspired by the Unix philosophy: do one thing well.

Built with minimalism, on a potato, for potatoes.

=====

END OF DOCUMENTATION

=====